



LINGUAGEM PYTHON

PROF. FERNANDO MARQUES

- A história do Python remonta aos anos 80, quando o desenvolvedor Guido van Rossum começou a trabalhar em uma nova linguagem de programação - Amoeba. Sua motivação era criar uma linguagem fácil de ler e escrever, com uma sintaxe clara e intuitiva.
- No início, Python foi desenvolvido como um projeto pessoal de van Rossum, mas logo atraiu a atenção e o interesse de outros programadores. Com o apoio da comunidade, a linguagem começou a evoluir e se tornar cada vez mais poderosa.
- Finalmente, em fevereiro de 1991, Guido van Rossum lançou a primeira versão pública do Python, marcando oficialmente o nascimento dessa linguagem. Desde então, Python tem percorrido um longo caminho, ganhando popularidade e se tornando uma ferramenta essencial para programadores de todo o mundo.
- Ao longo do desenvolvimento do Amoeba, Guido percebeu a necessidade de uma linguagem mais robusta e decidiu iniciar o desenvolvimento de uma nova linguagem, que inicialmente chamou de "Python", inspirado pelo seu amor pelo grupo de comédia britânico Monty Python.



Python é



linguagem de propósito geral



fácil e intuitiva



multiplataforma



batteries included



livre



organizada



orientada a objetos

- O interpretador Python já vem pré instalado em alguns Sistemas Operacionais, como Linux e IOS-Mac porém, no Windows devemos instalar por nossa conta.
- Para instalar o Python no Windows basta entrarmos no site www.python.org e lá irmos na guia downloads.



Donate



Search

GO

Socialize

About

Downloads

Documentation

Community

Success Stories

News

Events

```
# Python 3: List
>>> fruits = ['apple', 'banana', 'orange']
>>> loud_fruits = [fruit.upper() for fruit in fruits]
>>> print(loud_fruits)
['BANANA', 'APPLE', 'ORANGE']

# List and the enumerate function
>>> list(enumerate(fruits))
[(0, 'Banana'), (1, 'apple'), (2, 'orange')]
```

All releases

Source code

Windows

macOS

Other Platforms

License

Alternative Implementations

Download for Windows

Python 3.12.2

Note that Python 3.9+ cannot be used on Windows 7 or earlier.

Not the OS you are looking for? Python can be used on many operating systems and environments.

[View the full list of downloads.](#)

Python is a programming language that lets you work quickly and integrate systems more effectively. [>>> Learn More](#)

■ Instalando o Python 3 no Windows

- Para instalar o Python no seu sistema operacional Windows, você precisa baixar o instalador. Acesse o site oficial neste link (<https://www.python.org/downloads/>) e clique em download
- Faça o download do instalador executável do Windows (32 ou 64 bits) e clique duas vezes nele para iniciar o assistente de instalação do python
- O processo de instalação é bem simples.
- 1.- Marque a opção "Add Python to PATH"
- 2.- Clique em "Install Now"
- Para instalar o PIP no Windows: **py -m ensurepip --default-pip**

- Caso o pip não funcione: **py -c "import os, sys; print(os.path.join(sys.prefix, 'Scripts'))"** daí copie o path e cole nas variáveis de ambiente

- Abre um terminal LINUX

Instalação do PIP no LINUX

- `sudo apt install python3-pip`
- `sudo apt-get update`

- Para verificar se a instalação está ok, digite o seguinte comando: **python --version**
- Agora digite **pip --version**

■ Desenvolvimento de scripts de testes e praticidade

- Diversas IDEs (Ambiente de Desenvolvimento Integrado, do inglês Integrated Development Environment) surgiram ou se adaptaram para atender às necessidades de programadores.
- Um exemplo é o Jupyter Notebook, uma aplicação web open-source que nos dá suporte para criação e compartilhamento de documentos que podem conter códigos, equações, e scripts de visualização de dados. Outra IDE muito utilizada e madura é o Pycharm, que conta com versão paga, gratuita para a comunidade e estudantil (esta última com mais funcionalidade que a gratuita).IDLE também se encaixa na descrição.
- Utilizaremos o **VSCode**: uma ferramenta gratuita, open-source, leve e também muito utilizada na comunidade de desenvolvedores.

■ Estrutura de Repetição **FOR**

```
1 num = [1, 2, 3, 4]
2 for n in num:
3     print(2 + n)
4
```

```
2+1=3
2+2=4
2+3=5
2+4=6
```

```
peessoas = ['Pedro', 'Maria', 'Jaime', 'Joana']
for pes in pessoas:
    print('Bem-vindo(a) ' + pes + '!')

peessoas.append('Fernando')
print(peessoas)
```

- O comando **FOR**, basicamente é utilizado para percorrer listas (Arrays), repare que utilizamos a sintaxe reduzida do for, por ser mais elegante.
- Abra a pasta do seu novo projeto com o VS Code e crie um arquivo qualquer com a extensão py
- ❖ Faça um programa em python que defina uma lista com 4 nomes: Pedro, Maria, Jaime e Joana – Mostre então uma saudação para cada um deles, tipo: “Bem-vindo Pedro!”

■ f-string (formatted string literal)

- É a forma mais moderna, legível e rápida de formatar strings no Python.
- Introduzida no Python 3.6, ela se tornou o padrão recomendado para concatenar/interpoliar variáveis e expressões dentro de strings.
- Sintaxe Básica: Basta colocar a letra f (ou F) antes das aspas:

```
nome = "Ana"
idade = 25

print(f"Meu nome é {nome} e tenho {idade} anos.")
# Saída: Meu nome é Ana e tenho 25 anos.
```

```
nome = input("Digite seu nome: ")
print(f"Olá, {nome}!")
```

■ Interagindo com o usuário – INPUT

- O **input()** é uma função embutida do Python que permite ler dados digitados pelo usuário durante a execução do programa. Ele sempre retorna o valor como string (texto)

- Crie um programa em Python que peça ao usuário as seguintes informações:
- Nome completo - Idade (número inteiro) - Cidade onde mora - Profissão - Hobby favorito
- Depois, usando apenas f-strings, monte e imprima uma mensagem bem organizada e amigável com todas essas informações.
- A saída final deve ficar exatamente assim (substituindo com os dados digitados pelo usuário):

```

■ =====
■
■     MENSAGEM PERSONALIZADA
■
■     =====

```

C:\MyProjects\MyPyProjects\CursoPython\ExFStringInput

- Olá, meu nome é Ana Silva!
- Tenho 28 anos e moro em Londrina.
- Atualmente trabalho como Desenvolvedora de Software.
- No meu tempo livre, gosto muito de ler livros de ficção científica.

```

=====
MENSAGEM PERSONALIZADA
=====

Olá, meu nome é Fernando Marques!
Tenho 57 anos e moro em Londrina.
Atualmente trabalho como professor.
No meu tempo livre, gosto muito de cozinhar.

Resumo da sua vida: Fernando Marques é um(a) professor de 57 anos que mora em Londrina e adora cozinhar.
=====

```

- Resumo da sua vida: Ana Silva é uma Desenvolvedora de Software de 28 anos que mora em Londrina e adora ler livros de ficção científica.
- =====
- O programa deve limpar os espaços em branco extras que o usuário possa digitar (use `.strip()`). A mensagem deve ter o visual bonito com linhas de separação (como no exemplo acima). A idade deve aparecer corretamente como número (sem aspas).


```
■ nome = input("Digite seu nome completo: ").strip()
■ idade = int(input("Digite sua idade: "))
■ cidade = input("Digite a cidade onde você mora: ").strip()
■ profissao = input("Digite sua profissão: ").strip()
■ hobby = input("Digite seu hobby favorito: ").strip()
■
■ print("\n" + "=" * 40)
■ print("      MENSAGEM PERSONALIZADA")
■ print("=" * 40)
■ print(f"\nOlá, meu nome é {nome}!")
■ print(f"Tenho {idade} anos e moro em {cidade}.")
■ print(f"Atualmente trabalho como {profissao}.")
■ print(f"No meu tempo livre, gosto muito de {hobby}.")
■ print()
■ print(f"Resumo da sua vida: {nome} é um(a) {profissao} de {idade} anos que  
mora em {cidade}")
■ print(f"e adora {hobby}.")
■
■ print("=" * 40)
```

■ Usando Elementos Gráficos

- Abra sua pasta de projetos Python no VS-Code e crie um arquivo chamado **ex1.py**
- **(C:\MyProjects\MyPyProjects\exPyQt01\ex1.py)**

Para instalar o PyQt6, vc precisa ter o Python e o PIP – digite: **pip install PyQt6**

No Linux: **sudo apt update** e depois: **sudo apt install python3-pyqt6**

- `import sys`
- `from PyQt6.QtWidgets import QApplication, QWidget, QPushButton, QLabel`
- `from PyQt6.QtCore import Qt`
- `from PyQt6.QtGui import QCursor`
- `def funcao1():`
 - `label.setText('Botão pressionado')`
 - `label.adjustSize()`
- `app = QApplication(sys.argv)`
- `janela = QWidget()`

```
■ janela.resize(800,600)
■ janela.setWindowTitle('Primeira Janela')
■
  btn = QPushButton('Botao 1', janela)
■ btn.setGeometry(100, 100, 150, 80)
■ btn.clicked.connect(funcao1)

■ btn.setStyleSheet("""
■     QPushButton {
■         background-color: #0078D7;    /* azul mais bonito */
■         color: white;
■         border: none;                  /* importante! */
■         padding: 8px 16px;
■         border-radius: 4px;
■     }
```



```
■ QPushButton:hover {  
■     background-color: #106EBE;  
■ }  
■  
■ QPushButton:pressed {  
■     background-color: #005A9E;  
■ }  
■ """)  
■ btn.setCursor(Qt.CursorShape.PointingHandCursor)
```

- 
- `label = QLabel("Texto Qualquer", janela)`
 - `label.move(400,100)`
 - `label.setStyleSheet('font-size:30px')`
 - `janela.show()`
 - `app.exec()`

- Depois de vermos como funciona a parte gráfica diretamente no código, vamos utilizar uma ferramenta Gráfica para nos auxiliar.
- **Para instalar o PySide6:**
- **PySide6 x PyQt6**
- **O PyQt é independente e roda só até o Python 3.9, já o PySide é da própria Qt e roda no 3.12.**

WINDOWS

- # Abra o Prompt de Comando (terminal do VS-Code) ou PowerShell e crie o ambiente virtual:
- **`python -m venv meu_ambiente`**
- **`meu_ambiente\Scripts\activate`**

Principais razões para usar ambiente virtual

Motivo	Instalação Direta (Global)	Ambiente Virtual (venv)
Conflito de versões	Um projeto pode quebrar o outro	Cada projeto tem suas próprias versões
Segurança do Python do sistema	Pode estragar ferramentas do Windows ou outros programas	Não afeta nada fora do projeto
Facilidade de compartilhar	Difícil (outra pessoa precisa da mesma versão)	Fácil (basta copiar o <code>requirements.txt</code>)
Limpeza e organização	Tudo fica misturado no Python principal	Cada projeto fica isolado e limpo
Atualizações seguras	Atualizar um pacote pode quebrar tudo	Você atualiza só o que precisa, sem risco

- **LINUX**

Primeiro vc deverá habilitar o Ambiente Virtual no Linux:

```
apt install python3.12-venv
```

Agora digite:

```
python3 -m venv meu_ambiente
```

```
source meu_ambiente/bin/activate
```

```
sudo apt install libxcb-cursor0
```

o UV

UV é uma ferramenta moderna e extremamente rápida para gerenciar pacotes e projetos **Python**.

Criada pela **Astral**, a mesma empresa que criou o **Ruff** (o linter/formatter mais rápido do Python).

Alternativa Windows (+moderna +rápida) - Esse é o método oficial recomendado pela Astral.

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

```
.venv\Scripts\activate - Para ativar
```

Alternativa Linux

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

ou caso não tenha o curl

```
wget -qO- https://astral.sh/uv/install.sh | sh
```

USO: uv venv

```
.venv\Scripts\activate
```

- 3. Dica importante
- Se você usa VS Code, é muito fácil ver qual ambiente está sendo usado:
 1. Pressione **Ctrl + Shift + P**
 2. Digite "**Python: Select Interpreter**"
 3. Ele vai mostrar a lista dos ambientes que o VS Code encontrou
- Rode então: **pip install PySide6**
- Para abrir: **pyside6-designer**

Para instalar o Qt Designer:

<https://build-system.fman.io/qt-designer-download>

■ Criando a sua Janela Principal

- Instalado o Qt Designer, execute-o e crie a sua **MainWindow**
- Remova então o **MenuBar** e o **StatusBar** (não utilizaremos agora) – Clique com o direito em cima e embaixo e remova.
- Na caixa de pesquisas procure por **frame** e insira dois frames na tela e deixe-os com Lay Out Horizontal;
- Mude o nome do frame da esquerda para **left_container** e o da direita para **main_container**
- Insira mais 3 frames no **main_container** com lay out Vertical.

Renomeie os frames para **frm_header** – **frm_main** – **frm_footer**

Insira um **PushButton** e uma **Label** no **frm_header** e deixe com lay out horizontal;

Insira também uma **Label** no **frm_footer** e mude o texto para **Padoka**

Renomeie o **PushButton** para btn_Alternar

Deixe o **frm_main** em sizePolicy, na opção Política vertical **Expanding**

Vamos colocar então 2 Frames no lado esquerdo e deixá-los com lay out vertical;

Insira um Tool Box no frame de baixo (onde colocaremos os botões) - lay out horizontal;

Um Label no frame de cima, colocando o texto com o nome da empresa (ou logo): Padoka - lay out vertical;

No Object Inspector, em left-container, deixe a largura do **Maximum Size** como 200;

Insira 4 **PushButton** na **toolBox** e renomeie para **Home - Cadastrar - Contatos - Sobre**

e aplique um lay out vertical;

Embaixo do **Sobre**, insira um **Vertical Spacer**

Criando as Pastas

Insira 1 **Stacked Widget** no **frmMain** e lay out vertical no frmMain;

Renomeie o Stacked Widget para **Pages** e as páginas para **pg_home** e **pg_cadastrar**

Clique com o direito no **Pages** e insira mais duas páginas e renomeie-as para **pg_contatos** e **pg_sobre**;

Inserindo Imagens

Aqui entra uma questão, vc até pode inserir imagens .png ou .jpg diretamente no projeto, porém na distribuição elas poderão sumir, então o correto é inserir via **recursos**:

- Abra seu projeto no **Qt Designer**
- Abra o **Navegador de Recursos**
- Clique na **chave** ou **lápiz**
- Clique em **Novo arquivo de recurso**
- Dê um nome com extensão **.qrc** (ex. myrecursos.qrc)
- Salvar
- Clique em **Adicionar Prefixo** e insira suas imagens.

Insira uma **Label** na **pg_home** e renomeie para **logo**;

Clique com o direito no **logo** e em **Alterar Texto Rico (Change Rich Text)**

Clique então em **Inserir Imagem** e insira o **LOGO**

Página Cadastrar

Insira uma **Tab Widget** na página Cadastrar deixando o lay out vertical;
Coloque **Cadastro** na propriedade **CurrentTabText** e **Empresas** na 2ª aba;

Insira uma **Table Widget** dentro da **Tab Widget**;

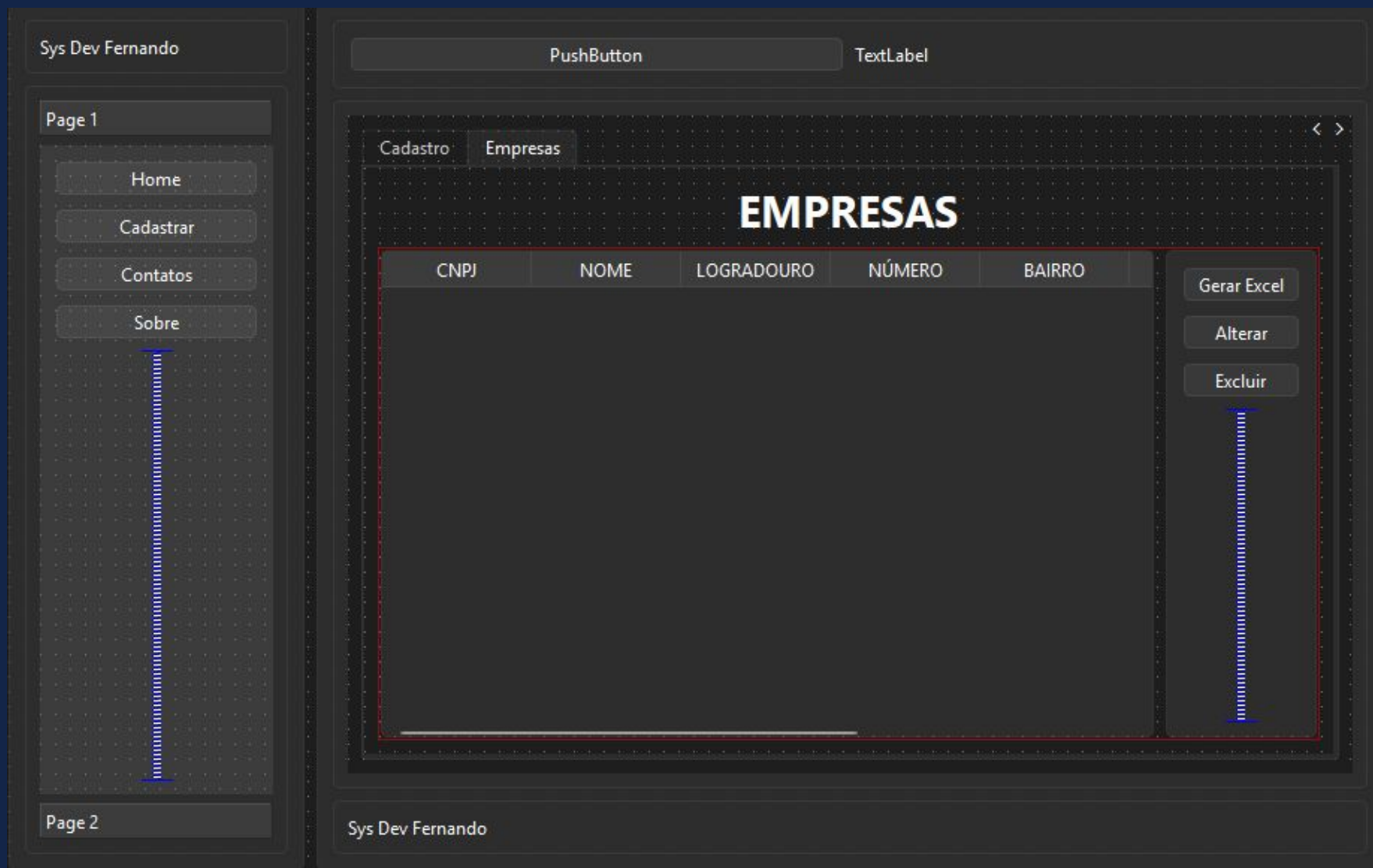
Ao lado da table insira um **Frame** e em cima, uma Label com o título EMPRESAS, tamanho 22
Negrito Centralizado;

No frame insira 3 botões: **Gerar Excel - Alterar e Excluir** e já renomeie no Object Inspector e lay out vertical;

Mantenha selecionado o Frame e selecione a Tab deixe o lay out horizontal;

Coloque um espaçador vertical embaixo do excluir.

Sua tela deverá estar com essa aparência:



Na Aba Empresas:

Para adicionar as colunas, basta dar 2 cliques na tabela e add... Adicione:

NOME - LOGRADOURO - NÚMERO - BAIRRO - MUNICÍPIO - TELEFONE - E-MAIL

Na Aba Cadastros:

Insira um **Frame** e dentro dele uma **Label** e 11 **Line Edit's** no Frame. Coloque os placeholders e distribua-os (use a distribuição em grade no final), como segue:

The screenshot shows a Qt Designer window with two tabs: 'Cadastro' and 'Empresas'. The 'Empresas' tab is active, displaying a form layout on a grid background. The form is titled 'Empresa' and contains the following fields:

CNPJ	Nome Empresarial	
Logradouro		
Número	Complemento	Bairro
Município	UF	CEP
Telefone	email	

Insira um **PushButton** embaixo do **Frame** e aplique um **layout vertical**. Clique com o direito no pushbutton e deixe alinhado horizontalmente. Selecione a **Label** e clique com o direito e **Alterar texto rico (chance rich text)** e centralize.

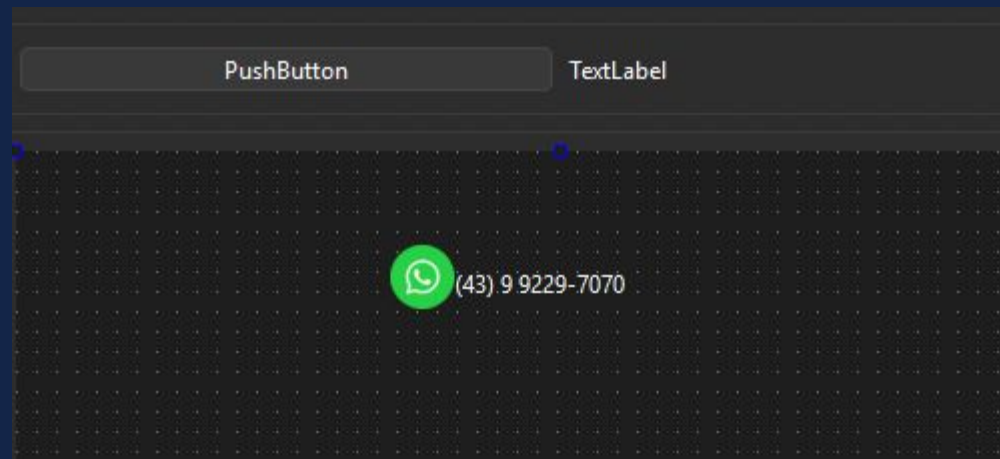
The image shows a Java Swing window titled "Cadastro" with a tab labeled "Empresas". The window contains a form for company registration. The form has a title "Empresa" and several input fields arranged in a grid. At the bottom right of the form is a "PushButton".

Empresa		
CNPJ	Nome Empresarial	
Logradouro		
Número	Complemento	Bairro
Município	UF	CEP
Telefone	email	

PushButton

Página Contatos

Insira uma **Label**, acesse o texto rico e insira o telefone da empresa, email, insta e YT...
Coloque o título (CONTATOS)



Página Sobre

Insira duas **Label**, a primeira escreva **SOBRE** e na 2ª acesse o texto rico e insira o texto:

Este sistema faz consulta do CNPJ utilizando API da Receita Federal e faz o cadastro em um banco de dados. O objetivo deste sistema é ensinar como utilizar Python e o QT para desenvolver aplicações modernas e funcionais.

Com a Label selecionada, procure no editor de propriedades por **Wrap** e marque a opção **wordWrap**.

Clique fora da Label e aplique layout vertical.

Vamos aplicar então um background-color para a nossa aplicação:

Clique com o direito em **MainWindows** em seguida em **AlterarFolhaEstilo;**
Adicionar Cor e **background-color**.

Clique com o direito em **centralwidget** em seguida em **AlterarFolhaEstilo** e insira:

```
*{  
    border: none;  
}
```

```
QLabel{  
    color: rgb(255, 255, 255);  
}
```

Selecione o **frame_2** - é o que envolve os botões e páginas - Nas folhas de estilo dele insira:

```
QFrame{  
    background-color: rgb(65, 65, 65);  
}
```

```
QToolBox{  
    text-align: left;  
    background-color: rgb(228, 254, 255);  
}
```

```
QToolBox::tab{  
    border-radius: 5px;  
    text-align: left;  
    background-color: rgb(194, 232, 255);  
}
```

Selecione o **Tool Box** (do frame_2) - Nas folhas de estilo dele insira:

```
QPushButton:hover {  
    background-color: rgb(65, 65, 65);  
    border-top-left-radius: 15px;  
}
```

```
QPushButton {  
    color: rgb(255, 255, 255);  
}
```




Selecione o botão **Home** mude seu nome para **btn_home** e coloque sua altura mínima em 30 e a **font-size** em 11;

FAÇA ISSO PARA CADASTRAR - CONTATOS E SOBRE

Ainda posicionado em **Home** - Mude o **Cursor** para **Mão Apontando** (Repita para os outros botões).



AQUI FICA UM GAP

FALTAM N CONFIGURAÇÕES ESTÉTICAS DA NOSSA INTERFACE

EXECUTANDO O SISTEMA

Vamos criar primeiro o nosso arquivo [main.py](#)

No seu ambiente virtual que já deve ter a biblioteca PySide6, vamos instalar os pacotes Requests e o Pandas (gerar o arquivo excel):

```
pip install requests
```

```
pip install pandas
```

E também o openpyxl (dependência do pandas para gerar o xlsx)

```
pip install openpyxl
```

O próximo passo é converter alguns arquivos para o Python:

```
pyside6-rcc myrecursos.qrc -o myrecursos_rc.py
```

```
pyside6-uic main_window.ui -o ui_main.py
```

Certo! Agora vamos ao arquivo `main.py`:

```
from PySide6.QtCore import QApplication
from PySide6.QtGui import QIcon
from PySide6.QtWidgets import (QApplication, QMainWindow)
from ui_main import Ui_MainWindow
import sys

class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.setupUi(self)
        self.setWindowTitle("SysDev - Sistema de cadastro de empresas")
        appIcon = QIcon(u"")
        self.setWindowIcon(appIcon)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    app.exec()
```

Aqui entra a POO









OBRIGADO

PESSOA@EXAMPLE.COM